NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 4: Functions, Decomposition, and Abstraction

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 4 at a glance

Lecture 10

Defining and calling functions

- function anatomy;
- parameters, arguments, and calls;
- return values and reuse.

Lecture 11

Interfaces and trustworthy functions

- keyword/default arguments;
- local scope and explicit inputs;
- docstrings, contracts, and tests.

Lecture 12

Decomposition and abstraction

- one clear job per function;
- readable engineering workflows;
- reusable calculation libraries.

Identify a task → Define its interface → Implement and test it → Compose trusted pieces

NAZARBAYEV UNIVERSITY

LECTURE 10

Defining and Calling Functions

Turning repeated engineering calculations into named, reusable, and testable program components

NAZARBAYEV UNIVERSITY

Lecture 10 learning goals

By the end of this lecture, you should be able to:

1. explain why meaningful repeated work should become a function;
2. define a function using `def`;
3. identify its name, parameters, body, and `return`;
4. distinguish a definition from a call;
5. distinguish parameters from arguments;
6. reuse a returned value and distinguish it from printed output;
7. trace and repair a short function call;
8. verify a function with a hand-checkable case.

From long scripts to small reusable tools

NAZARBAYEV UNIVERSITY

Retrieval warm-up: where is the repeated idea?

```
force_1 = 5000        # N
area_1 = 0.002        # m^2
stress_1 = force_1 / area_1
force_2 = 8000        # N
area_2 = 0.003        # m^2
stress_2 = force_2 / area_2
force_3 = 12000       # N
area_3 = 0.004        # m^2
stress_3 = force_3 / area_3
```

Discuss

1. Which operation repeats?
2. What changes each time?
3. What should the tool be called?
4. What inputs must it receive?

Pairs - 3 min

NAZARBAYEV UNIVERSITY

Opening problem: a correct script that is hard to grow

The current script

It converts an area, calculates stress, calculates factor of safety, classifies the design, and prints a report.

- What happens when we evaluate ten designs?
- What happens when the stress formula changes?
- Where should we test the unit conversion?
- Can another program reuse only the stress calculation?

The real problem

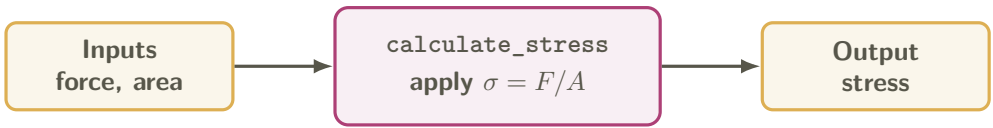
The code may run, yet the engineering ideas are hidden inside one long sequence of statements.

Week 4 response

Give each meaningful task a name and a clear interface.

NAZARBAYEV UNIVERSITY

A function is a named input–process–output unit



For the caller

The name and interface explain how to use the tool.

Inside the function

Python executes the statements that implement the calculation.

NAZARBAYEV UNIVERSITY

You have already called many functions

```
print("ENG 101")
length = len("sensor")
magnitude = abs(-12.5)
largest = max(3, 8, 5)
kind = type(3.14)
```

Built-in functions

Python already provides named tools. Each has:

- a function name;
- arguments inside parentheses;
- defined behavior;
- sometimes a returned value.

Now define your own

Your first engineering function

```python
def calculate_stress(force, area):
    stress = force / area
    return stress

result = calculate_stress(5000, 0.002)
print(f"Stress = {result / 1e6:.2f} MPa")
```

Output

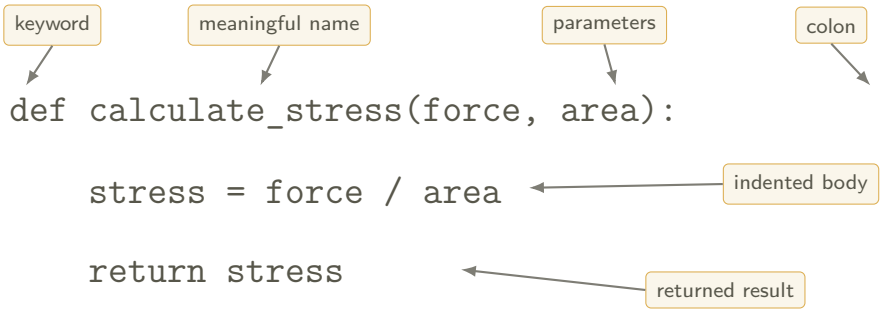
```
Stress = 2.50 MPa
```

Unit meaning

With force in N and area in m^2, the returned stress is in Pa.

NAZARBAYEV UNIVERSITY

Anatomy of a function definition



Python uses indentation to define the function body

Every statement belonging to the function must be indented consistently.

NAZARBAYEV UNIVERSITY

Definition and call are different events

Definition: create the tool

```
def calculate_current(voltage, resistance):
    return voltage / resistance
```

Python records how the function should behave. The body is not executed yet.

Call: use the tool

```
current = calculate_current(12, 4)
print(current)
```

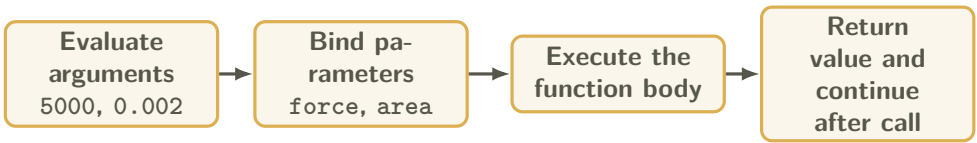
The call passes values, runs the body, and receives the returned result.

Common beginner surprise

A function can be defined perfectly and still appear to “do nothing” because it was never called.

NAZARBAYEV UNIVERSITY

What happens during a function call?



Control flow

Execution temporarily moves into the function. When `return` is reached, execution moves back to the calling statement with a value.

Parameters and arguments

```
def calculate_stress(force, area):
    return force / area

stress = calculate_stress(5000, 0.002)
```

In the definition

`force` and `area` are **parameters**: local names used by the function.

In the call

5000 and 0.002 are **arguments**: values supplied by the caller.

NAZARBAYEV UNIVERSITY

Positional arguments: order carries meaning

Intended call

```
stress = calculate_stress(5000, 0.002)
```

`force = 5000`, `area = 0.002`

Reversed call

```
stress = calculate_stress(0.002, 5000)
```

Python can still divide the numbers, but the engineering meaning is wrong.

Important lesson

A syntactically valid function call can be physically meaningless. The caller remains responsible for order, units, and assumptions.

NAZARBAYEV UNIVERSITY

Keyword arguments make the interface visible

```
stress = calculate_stress(
    force=5000,
    area=0.002
)
```

Advantages

- the meaning is visible at the call site;
- argument order becomes less ambiguous;
- long engineering calls are easier to review;
- a unit-bearing parameter name can guide the user.

Readable calls prevent mistakes

NAZARBAYEV UNIVERSITY

Returned values support later calculations

```
def calculate_current(voltage, resistance):
    return voltage / resistance

current = calculate_current(12, 4)
power = 12 * current
energy = power * 3600

print(f"Current = {current:.2f} A")
print(f"Energy = {energy:.0f} J")
```

Return enables reuse

A returned value can be:

- stored;
- printed;
- used in an expression;
- passed to another function.

NAZARBAYEV UNIVERSITY

`print()` and `return` answer different needs

	`print()`	`return`
Purpose	communicate information to a human	send a value back to the caller
Reusable in calculation?	not by itself	yes
Ends the function call?	no	yes, when executed
Typical location	reporting layer	calculation or decision function

```
def calculate_power(voltage, current):
    return voltage * current

power = calculate_power(12, 3)
print(f"Power = {power:.1f} W")
```

The `None` surprise

```
def calculate_stress_bad(force, area):
    stress = force / area
    print(stress)

value = calculate_stress_bad(5000, 0.002)
print("Stored value:", value)
```

Output

```
2500000.0
Stored value: None
```

Why?

The function displayed a number but did not return one. A function without an executed value-returning statement returns `None`.

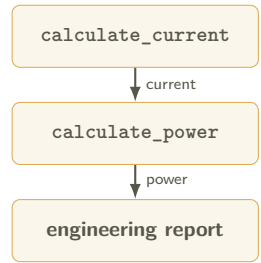
NAZARBAYEV UNIVERSITY

Compose small functions into a calculation chain

```
def calculate_current(voltage, resistance):
    return voltage / resistance


def calculate_power(voltage, current):
    return voltage * current

current = calculate_current(12, 4)
power = calculate_power(12, current)
```



Readable workflow

Each line names an engineering step rather than exposing every arithmetic detail.

NAZARBAYEV UNIVERSITY

Engineering formulas make natural functions

```
def calculate_stress(force, area):
    return force / area


def calculate_strain(delta_length,
                     original_length):
    return delta_length / original_length
```

```
def calculate_kinetic_energy(mass,
                             velocity):
    return 0.5 * mass * velocity**2


def calculate_density(mass, volume):
    return mass / volume
```

Function-selection question

Can the task be described as one meaningful calculation with clear inputs and a clear output?

A function call is an expression

```
def calculate_velocity(distance, time):
    return distance / time

speed_m_s = calculate_velocity(100, 9.58)
speed_km_h = calculate_velocity(100, 9.58) *
    3.6

print(speed_m_s)
print(speed_km_h)
```

Why this matters

The function call evaluates to its returned value, so it can appear wherever a compatible value can appear.

Still prefer clarity

Store important intermediate engineering quantities when a direct expression would hide units or meaning.

NAZARBAYEV UNIVERSITY

Activity: design the interface before the body

A device with power P operates for time t. We need energy in joules, $E = Pt$.

Write only these four items

1. a meaningful function name;
2. parameter names that reveal units;
3. the expected returned quantity and unit;
4. one hand-checkable call.

One possible interface

`calculate_energy_j(power_w, time_s)`

Returns energy in joules.

Test: `calculate_energy_j(10, 2)` should return 20.

Individual then compare - 4 min

NAZARBAYEV UNIVERSITY

Live-coding ladder: grow one function carefully

Stage 1: formula

```
def convert_kpa_to_pa(pressure_kpa):
    return pressure_kpa * 1000
```

Stage 2: call

```
pressure_pa = convert_kpa_to_pa(250)
```

Stage 3: communicate

```
print(f"Pressure = {pressure_pa:.0f} Pa")
```

Stage 4: modify

Change the call to 85 kPa. Predict the result before running. Then add a second call without changing the function body.

NAZARBAYEV UNIVERSITY

Common first-function syntax errors

Missing colon

```
def stress(f, a)
    return f / a
```

Missing indentation

```
def stress(f, a):
return f / a
```

Missing parentheses

```
value = stress 5000, 0.002
```

Repair strategy

Check the header, colon, indentation, parentheses, number of arguments, and exact function name – one issue at a time.

NAZARBAYEV UNIVERSITY

Debugging clinic: why does this script fail?

```
def calculate_power(voltage, current):
    power = voltage * current
    return power

print(power)
```

Diagnosis

power is a local variable inside the function. The function is also never called.

Repair

```
power = calculate_power(12, 3)
print(power)
```

Explain before fixing - 3 min

NAZARBAYEV UNIVERSITY

First test: use numbers you can verify by hand

```
def calculate_stress(force, area):
    return force / area

expected = 5
actual = calculate_stress(10, 2)

print("Expected:", expected)
print("Actual:", actual)
print("Pass:", actual == expected)
```

Why simple cases first?

- the expected answer is independent of the code;
- mistakes are easier to locate;
- the function can be checked separately from the full program.

Small test, strong evidence

Lecture 10 exit ticket

Without running Python

For a function that calculates density $\rho = m/V$, write:

1. the function header;
2. one return statement;
3. one call using $m = 2.7\,\mathrm{kg}$ and $V = 0.001\,\mathrm{m}^3$;
4. the expected result and unit.

Today in one sentence

A function packages one meaningful task behind a name, receives arguments through parameters, and returns a value that the rest of the program can use.

Individual - 3 min

NAZARBAYEV UNIVERSITY

LECTURE 11

Function Interfaces, Scope, Documentation, and Tests

Designing functions whose use, assumptions, and evidence of correctness are visible to other engineers

NAZARBAYEV UNIVERSITY

Lecture 11 learning goals

By the end of this lecture, you should be able to:

1. describe a function interface using its name, parameters, and returned value;
2. use positional, keyword, and default arguments appropriately;
3. explain that variables created inside a function are normally local;
4. pass required engineering quantities explicitly instead of hiding dependencies;
5. return two related results when this improves the interface;
6. write a function containing a decision or loop;
7. write a docstring that records purpose, units, output, and assumptions;
8. interpret a function specification as a contract;
9. use simple assertions and tolerances to test returned values;
10. diagnose wrong argument order, missing returns, and scope errors.

Retrieval warm-up: predict every printed line

```
def calculate_power(voltage, current):
    power = voltage * current
    print("Inside:", power)
    return power

result = calculate_power(12, 3)
print("Outside:", result)
```

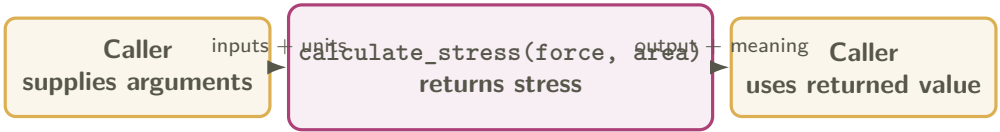
Trace

1. Which line runs first?
2. When does control enter the function?
3. What value is returned?
4. Is the inside variable available afterward?

Individual - 3 min

NAZARBAYEV UNIVERSITY

The interface is the visible agreement with the caller



A useful interface answers

What must the caller provide? What does the function promise to return? Which units and assumptions make that promise meaningful?

NAZARBAYEV UNIVERSITY

Three ways to supply arguments

Positional

```
stress = calculate_stress(
    5000, 0.002
)
```

Compact, but order carries meaning.

Keyword

```
stress = calculate_stress(
    force=5000,
    area=0.002
)
```

More explicit at the call site.

Default

```
def status(fos,
           required=2.0):
    ...
```

A common value is supplied unless overridden.

NAZARBAYEV UNIVERSITY

Keyword arguments clarify long engineering calls

```
def calculate_reynolds_number(
        density, velocity, diameter, viscosity):
    return density * velocity * diameter /
    viscosity

re = calculate_reynolds_number(
    density=1000,
    velocity=2.0,
    diameter=0.05,
    viscosity=0.001
)
```

Review advantage

A reader can inspect the physical meaning of each number without memorizing parameter order.

Still missing

The call does not show units unless parameter names or documentation make them clear.

NAZARBAYEV UNIVERSITY

Default values express a normal policy – not a universal truth

```
def classify_design(factor_of_safety,
                    required_fos=2.0):
    if factor_of_safety >= required_fos:
        return "acceptable"
    return "not acceptable"

print(classify_design(2.5))
print(classify_design(2.5, required_fos=3.0))
```

Interpretation

The first call uses the default requirement of 2.0. The second call explicitly applies a stricter requirement.

Engineering responsibility

A default should be documented and should not silently replace the actual design code or project requirement.

NAZARBAYEV UNIVERSITY

Scope answers: where does a name exist?

Local scope

Parameters and variables assigned inside a function normally belong to that function call.

- they help the function perform its job;
- they reduce accidental interference;
- they usually disappear when the call finishes;
- the caller receives only what is returned.

Think of a workbench

A function call gets a temporary workbench containing its parameters and local variables. When the task finishes, the workbench is cleared; the returned result is carried out.

NAZARBAYEV UNIVERSITY

Local variables stay inside their function

```
def calculate_stress(force, area):
    stress = force / area
    return stress

result = calculate_stress(5000, 0.002)
print(result)
print(stress)      # What happens here?
```

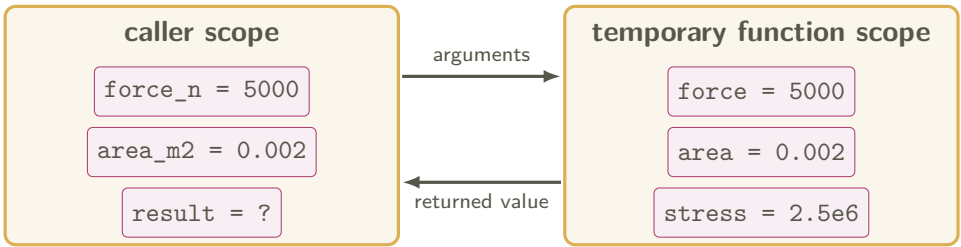
Result

`stress` is local to the function. The final line raises a `NameError`.

Correct communication

Store and use the returned value: `result`.

A simple call-frame picture



The names may differ

The caller's `force_n` can become the parameter `force`. Parameter binding transfers the value, not the caller's variable name.

NAZARBAYEV UNIVERSITY

Two functions may use the same local name safely

```
def calculate_stress(force, area):
    result = force / area
    return result


def calculate_current(voltage, resistance):
    result = voltage / resistance
    return result
```

No conflict

Each call creates its own local `result`. Local scope lets functions be developed and tested independently.

Local names protect components

NAZARBAYEV UNIVERSITY

Prefer explicit inputs to hidden dependencies

Hidden dependency

```
area = 0.002

def calculate_stress(force):
    return force / area
```

The function needs area, but its interface does not say so.

Explicit dependency

```
def calculate_stress(force, area):
    return force / area
```

The caller can see, control, document, and test both inputs.

Design rule

A good engineering function should usually receive the quantities it needs through parameters.

Returning two related results

```
def stress_and_fos(force, area, strength):
    stress = force / area
    factor_of_safety = strength / stress
    return stress, factor_of_safety

stress, fos = stress_and_fos(
    force=10000,
    area=0.0005,
    strength=250e6
)
```

Multiple return values

Python packages the results together, and the caller can unpack them into separate names.

Scope note

Tuples are studied formally next week. For now, focus on the interface: two related outputs are returned together.

Functions can contain engineering decisions

```
def classify_temperature(temp_c,
                         low_c,
                         high_c):
    if temp_c < low_c:
        return "too low"
    elif temp_c <= high_c:
        return "normal"
    else:
        return "too high"
```

One clear job

The function classifies a temperature. It does not also ask for input, calculate energy, or print a long report.

Test the boundaries

What should be returned at exactly `low_c` and exactly `high_c`?

Functions can contain loops without exposing the loop

```
def count_digits(code):
    count = 0
    for character in code:
        if character.isdigit():
            count += 1
    return count

number = count_digits("PUMP-107A")
```

Abstraction begins here

The caller asks for `count_digits(code)`. The loop is an implementation detail inside the function.

Expected result

The code contains three digit characters, so the function returns 3.

NAZARBAYEV UNIVERSITY

A docstring records the engineering meaning

```
def calculate_stress(force, area):
    """
    Return uniform axial stress in pascals.

    force: applied axial force in newtons
    area: cross-sectional area in square meters

    Assumption: load is axial and stress is
    uniformly distributed over the area.
    """
    return force / area
```

A useful engineering docstring states

- purpose;
- input meaning and units;
- output meaning and units;
- important assumptions.

A specification is a contract

Caller assumptions

Conditions that must be satisfied when using the function:

- acceptable input types;
- required units;
- valid ranges;
- model assumptions.

Function guarantee

What the function promises when the assumptions hold:

- returned quantity;
- output unit;
- accuracy or tolerance;
- special failure indicator, if any.

Contracts do not remove responsibility

A function may satisfy its specification while the specification itself is unsuitable for the real engineering problem.

NAZARBAYEV UNIVERSITY

Optional type hints can strengthen readability

```
def calculate_stress(
        force: float,
        area: float) -> float:
    """Return stress in Pa from N and m^2."""
    return force / area
```

What they do

Type hints document expected kinds of values and help editors analyze code.

What they do not do

They do not encode physical units, prove correctness, or automatically enforce engineering assumptions.

Optional in ENG 101

NAZARBAYEV UNIVERSITY

Simple assertions make expectations executable

```
def calculate_current(voltage, resistance):
    return voltage / resistance

assert calculate_current(12, 4) == 3
assert calculate_current(0, 10) == 0
```

Behavior

If the condition is true, execution continues silently. If false, Python raises `AssertionError`.

Why functions help testing

Clear inputs and outputs allow one component to be checked independently of the full script.

NAZARBAYEV UNIVERSITY

Test more than one convenient example

For `classify_temperature(temp, low, high)`:

Test category	Example	Expected result
representative low	$5, 10, 30$	too low
lower boundary	$10, 10, 30$	normal
representative normal	$20, 10, 30$	normal
upper boundary	$30, 10, 30$	normal
representative high	$35, 10, 30$	too high
questionable limits	$20, 30, 10$	assumptions violated

Testing question

Which behavior is guaranteed, and which invalid cases must the caller prevent or document?

Floating-point functions need tolerance-based tests

```
def celsius_to_kelvin(temp_c):
    return temp_c + 273.15

actual = celsius_to_kelvin(25.0)
expected = 298.15

tolerance = 1e-9
assert abs(actual - expected) < tolerance
```

Avoid fragile equality

Many decimal values are represented approximately in binary floating point. Ask whether results are close enough for the test purpose.

Tolerance has meaning

Choose it from the numerical method, measurement resolution, or required engineering accuracy.

NAZARBAYEV UNIVERSITY

Common function failures and their clues

Symptom	Likely cause	First check
Nothing appears	function defined but not called	locate the call
`None` **used in arithmetic**	printed instead of returned	inspect `return`
Physically absurd number	wrong argument order or units	trace parameter binding
`NameError` **outside function**	local variable used externally	use returned value
Result changes unexpectedly	hidden global dependency	list explicit inputs
Code after a point never runs	`return` executed too early	trace control flow
Function difficult to test	too many unrelated responsibilities	split the job

NAZARBAYEV UNIVERSITY

Debugging activity: find four distinct problems

```
area_mm2 = 500

def bar_check(force, strength):
    area_m2 = area_mm2 / 1000
    stress = force / area_m2
    print(stress)

stress = bar_check(10000, 250e6)
fos = 250e6 / stress
print(fos)
```

Classify each issue

1. hidden dependency;
2. area conversion error;
3. unused parameter;
4. printing instead of returning.

Pairs - 5 min

Function interface checklist

Before implementation

- Can I state the job in one sentence?
- Is the name specific and meaningful?
- Are all required inputs parameters?
- Do parameter names reveal physical meaning?
- What exactly should be returned?

Before reuse

- Are units and assumptions documented?
- Have representative and boundary cases been tested?
- Can a teammate call it without reading the body?
- Is the output suitable for later calculations?
- Is failure reported honestly?

Lecture 11 exit ticket

Design a contract

For `calculate_density(mass_kg, volume_m3)`, write:

1. two assumptions;
2. the guarantee;
3. one representative test;
4. one invalid input that the current function does not handle.

Today in one sentence

A trustworthy function has an explicit interface, local implementation details, documented assumptions, a reusable returned value, and independent evidence from tests.

Individual - 4 min

NAZARBAYEV UNIVERSITY

LECTURE 12

Decomposition and Abstraction in Engineering Programs

Organizing a larger problem as a readable workflow of small functions with clear responsibilities

NAZARBAYEV UNIVERSITY

Lecture 12 learning goals

By the end of this lecture, you should be able to:

1. explain decomposition as dividing a larger problem into smaller tasks;
2. identify function candidates from an engineering procedure;
3. design functions with one clear responsibility;
4. represent data flow among functions;
5. explain abstraction through a clear interface;
6. separate calculation from input/output and reporting;
7. refactor a Week 3 search into a reusable function;
8. build a function-based engineering report;
9. test components before integration;
10. outline a reusable function library;
11. prepare for Assignment 2 using Moodle's authoritative instructions.

Organize a large problem as a workflow of small trusted functions

NAZARBAYEV UNIVERSITY

Retrieval warm-up: where would you draw the boundaries?

```
force_n = 10000
area_mm2 = 500
strength_mpa = 250
required_fos = 2.0

area_m2 = area_mm2 / 1e6
stress_pa = force_n / area_m2
stress_mpa = stress_pa / 1e6
strength_pa = strength_mpa * 1e6
fos = strength_pa / stress_pa
status = "acceptable" if fos >= required_fos else "not
    acceptable"

print(f"Stress = {stress_mpa:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
print(f"Status = {status}")
```

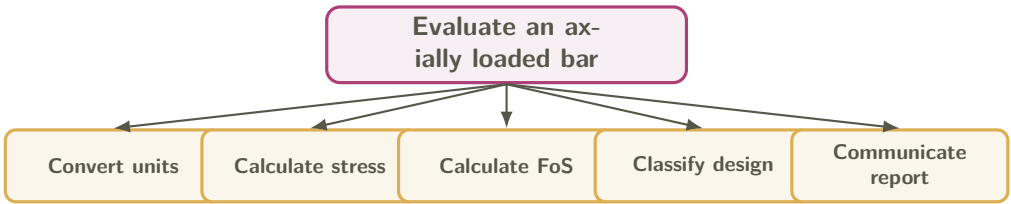
Mark possible functions

1. unit conversion;
2. stress calculation;
3. factor of safety;
4. classification;
5. report generation.

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Decomposition: turn one large task into smaller questions



A decomposition is useful when

each subtask can be understood, implemented, checked, and changed with limited dependence on the others.

The guiding question: what is the job of this function?

Focused function

`calculate_stress(force, area)`

- receives two quantities;
- applies one model;
- returns one result;
- can be tested independently.

Overloaded function

`do_everything()`

- asks for input and converts units;
- calculates several quantities;
- classifies, plots, saves, and prints;
- is difficult to reuse or diagnose.

Rule of thumb

If the job description needs several unrelated verbs joined by "and," consider splitting the function.

NAZARBAYEV UNIVERSITY

A decomposition table comes before code

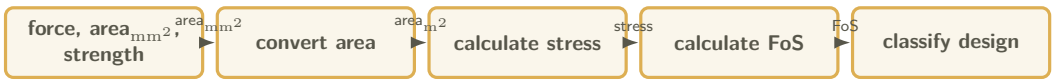
Function	Single responsibility	Returned value
convert_mm2_to_m2	convert area units	area in m^2
calculate_stress	apply $\sigma = F/A$	stress in Pa
calculate_fos	compute strength/stress	dimensionless FoS
classify_design	compare FoS with requirement	status string
format_design_report	create human-readable text	report string

Design before syntax

Names, inputs, outputs, and responsibilities can be reviewed before any function body is written.

NAZARBAYEV UNIVERSITY

Data flow connects the components



Make interfaces meet

The output of one function should have the meaning, type, and unit expected by the next function.

NAZARBAYEV UNIVERSITY

Top-down design and bottom-up confidence

Top-down: plan the system

1. state the engineering question;
2. list major steps;
3. assign one responsibility to each function;
4. define interfaces and data flow.

Bottom-up: build trust

1. implement one small function;
2. test it with a known case;
3. connect two trusted functions;
4. test the integrated workflow.

Do not integrate five untested functions and then debug the whole program at once

Component tests localize errors and make integration evidence stronger.

A well-decomposed program reads like an engineering procedure

```
area_m2 = convert_mm2_to_m2(area_mm2)
stress_pa = calculate_stress(force_n, area_m2)
fos = calculate_factor_of_safety(strength_pa,
                                 stress_pa)
status = classify_design(fos, required_fos)
report = format_design_report(stress_pa,
                              fos, status)
print(report)
```

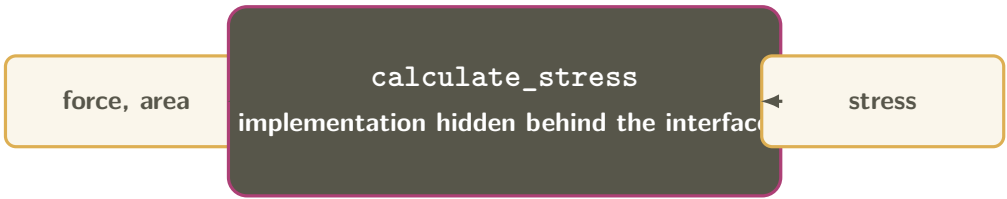
Read aloud

Convert the area, calculate stress, calculate factor of safety, classify the design, format the report, and print it.

Code mirrors reasoning

NAZARBAYEV UNIVERSITY

Abstraction: use the interface, hide the repeated detail



Abstraction preserves what is relevant

The caller needs the function's purpose, required inputs, returned output, and contract – not every internal statement on every use.

Implementation can change while the interface remains stable

Version A

Direct formula inside `calculate_stress(force, area)`.

Version B

Additional input checks and clearer error handling.

Version C

Vectorized NumPy implementation for many areas.

Caller code can remain

```
stress =
calculate_stress(force, area)
```

Condition

Each implementation must still satisfy the same documented contract.

NAZARBAYEV UNIVERSITY

Good abstraction depends on good names

Weak

```
def f(x, y):
    return x / y

z = f(a, b)
```

The reader must reconstruct the purpose.

Strong

```
def calculate_stress(force, area):
    return force / area

stress = calculate_stress(force, area)
```

The interface communicates the engineering operation.

Naming test

Can a teammate understand the main workflow without opening the function body?

NAZARBAYEV UNIVERSITY

Separate computation from input and reporting

Layer	Responsibility	Example
Input layer	read and convert user input	`float(input(...))`
Calculation layer	apply formulas and numerical algorithms	`calculate_stress(...)`
Decision layer	apply stated engineering rules	`classify_design(...)`
Presentation layer	format and display results	`format_report(...)`

Why separate layers?

The calculation can be reused in a notebook, automated test, graphical interface, or later project without rewriting the mathematics.

NAZARBAYEV UNIVERSITY

Refactor Week 3: package bisection as a reusable function

```
def square_root_bisection(value, tolerance=1e-6):
    """Return nonnegative square root approximation."""
    low = 0.0
    high = max(1.0, value)
    midpoint = (low + high) / 2

    while abs(midpoint**2 - value) >= tolerance:
        if midpoint**2 < value:
            low = midpoint
        else:
            high = midpoint
        midpoint = (low + high) / 2

    return midpoint
```

What moved inside?

- bracket variables;
- loop;
- midpoint updates;
- stopping test.

Assumption

`value` must be nonnegative.

The caller now reuses the numerical method

```
root_2 = square_root_bisection(2.0)
root_10 = square_root_bisection(10.0)
root_25 = square_root_bisection(
    25.0,
    tolerance=1e-10
)

print(root_2)
print(root_10)
print(root_25)
```

Abstraction benefit

Each call states the target and accuracy. The interval-halving details are written and tested once.

Next design question

How should the function report a negative input or a failure to converge? Later weeks introduce richer error handling.

Specify the numerical abstraction precisely

Assumptions

- `value` is an `int` or `float`;
- `value` is nonnegative;
- `tolerance` is positive;
- floating-point arithmetic is adequate.

Guarantee

Return a nonnegative number r such that

$$|r^2 - \texttt{value}| < \texttt{tolerance}.$$

Do not promise more

The contract controls the residual, not necessarily the exact error in r or every possible input condition.

NAZARBAYEV UNIVERSITY

Bar design functions: calculations and decision

```python
def convert_mm2_to_m2(area_mm2):
    return area_mm2 / 1e6


def calculate_stress(force_n, area_m2):
    return force_n / area_m2


def calculate_fos(strength_pa, stress_pa):
    return strength_pa / stress_pa
```

```python
def classify_design(fos,
                    required_fos=2.0):
    if fos >= required_fos:
        return "ACCEPTABLE"
    return "NOT ACCEPTABLE"
```

Focused responsibilities

Unit conversion, formula evaluation, and policy-based classification can be checked separately.

The integrated calculation function

```
def evaluate_bar(force_n, area_mm2, strength_pa,
                 required_fos=2.0):
    area_m2 = convert_mm2_to_m2(area_mm2)
    stress_pa = calculate_stress(force_n, area_m2)
    fos = calculate_fos(strength_pa, stress_pa)
    status = classify_design(fos, required_fos)
    return stress_pa, fos, status

stress_pa, fos, status = evaluate_bar(
    force_n=10000,
    area_mm2=500,
    strength_pa=250e6
)
```

Role of this function

It coordinates the calculation workflow and returns the engineering results. It does not decide how they must be displayed.

Orchestration function

Keep reporting separate from calculation

```
def format_bar_report(stress_pa, fos, status):
    stress_mpa = stress_pa / 1e6
    return (
        "BAR DESIGN REPORT\n"
        f"Stress: {stress_mpa:.2f} MPa\n"
        f"Factor of safety: {fos:.2f}\n"
        f"Status: {status}"
    )

report = format_bar_report(stress_pa, fos, status
    )
print(report)
```

Reuse advantage

The numerical results could instead be written to a file, plotted, or passed to another decision without rerunning the formulas.

Report honestly

"Acceptable" refers only to the stated simplified model and required FoS – not every possible failure mode.

NAZARBAYEV UNIVERSITY

Test pieces before testing the whole program

```
assert convert_mm2_to_m2(1_000_000) == 1
assert calculate_stress(10, 2) == 5
assert calculate_fos(100, 20) == 5
assert classify_design(2.0, 2.0) == "
    ACCEPTABLE"

stress, fos, status = evaluate_bar(
    force_n=10,
    area_mm2=2_000_000,
    strength_pa=20,
    required_fos=2.0
)
```

Test hierarchy

1. unit conversion;
2. each formula;
3. boundary decision;
4. integrated data flow;
5. physical interpretation.

Localize before integrating

NAZARBAYEV UNIVERSITY

A small engineering library collects reusable functions

For now: one notebook or file

```
# engineering_calculations.py

def calculate_stress(...):
    ...

def calculate_current(...):
    ...

def calculate_power(...):
    ...
```

Later: import the tools

```
from engineering_calculations import (
    calculate_stress,
    calculate_power
)
```

Modules are a later organizational step. The foundation is a set of focused, documented, tested functions.

NAZARBAYEV UNIVERSITY

Activity: decompose an energy-cost calculator

A device has power in watts, operating time in hours, and electricity price in KZT per kWh. The program must report energy in kWh and cost.

Design at least three functions

For each function, write:

1. name;
2. parameters and units;
3. returned quantity and unit;
4. one known test.

Possible decomposition

- `watts_to_kilowatts`;
- `calculate_energy_kwh`;
- `calculate_cost`;
- `format_energy_report`.

Small groups - 6 min

NAZARBAYEV UNIVERSITY

Pair design review: can another engineer use your function?

Builder

Choose mechanics, circuits, fluids, energy, or sensors. Propose four function interfaces and write one docstring.

- no function bodies at first;
- include units;
- state one assumption;
- identify one returned value.

Reviewer

Without seeing the implementation, answer:

- Can I call it correctly?
- Is any dependency hidden?
- Is the output unambiguous?
- What test would I run first?

Exchange and revise - 6 min

NAZARBAYEV UNIVERSITY

Assignment 2 launch: function-based problem solving

Course-aligned preparation

Expect to demonstrate that you can:

- identify appropriate functions;
- define clear parameters and returns;
- use meaningful names;
- document units and assumptions;
- test small components;
- combine functions into an engineering workflow.

Authoritative instructions

Use the Assignment 2 document on Moodle for the exact task, deadline, permitted resources, submission format, marking criteria, and AI-use rules.

Before submission

Be prepared to explain, reproduce, test, and modify every submitted function.

Week 4 scope boundary

Developed this week

- function definitions and calls;
- parameters, arguments, and returns;
- local scope;
- documentation and simple tests;
- decomposition and abstraction.

Developed later

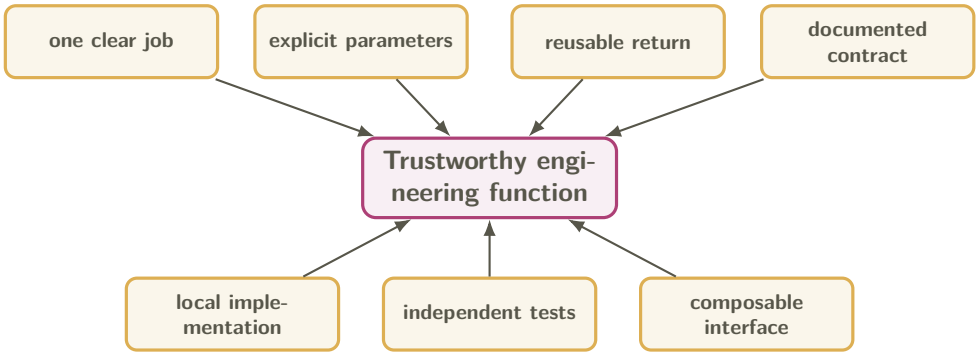
- tuples and lists: Week 5;
- recursion and dictionaries: Week 6;
- systematic testing and exceptions: Week 7;
- classes and objects: Weeks 8–9.

Why the boundary matters

Strong function design is the foundation for the later data structures, recursion, testing, and object-oriented programming topics.

NAZARBAYEV UNIVERSITY

Week 4 concept map



The larger idea

Decomposition creates structure; abstraction lets us use that structure without carrying every implementation detail in our heads.

Lecture 12 exit ticket

Decompose a projectile calculation

A program must compute height and velocity at a specified time and print a report. Propose:

1. two calculation functions;
2. one reporting function;
3. the data passed between them;
4. one test for each calculation.

Week 4 closing message

Functions are not merely syntax. They are a way to organize engineering thought into named, testable, reusable promises.

Individual - 4 min

Week 4 quick reference

Definition and call

`def f(x):`	define a function with parameter x
`return value`	send a result to the caller
`y = f(3)`	call the function and store its result
`f(x=3)`	use a keyword argument
`def f(x, tol=0.01):`	provide a default value
`return a, b`	return two related values

Engineering design habits

One clear job	keep each function focused
Explicit inputs	pass required quantities as parameters
Meaningful name	reveal the engineering operation
Docstring	state units, purpose, and assumptions
Simple test	use a case checked by hand
Small pieces	compose functions into a readable workflow

The four questions for every function

What does it need? What does it return? What assumptions must hold? How will we know it works?

NAZARBAYEV UNIVERSITY

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 4.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapter 7 (functions, decomposition, abstraction, documentation, and testing).
- J. V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Chapter 4, Sections 4.1–4.2.

Primary preparation for students

Run every function, predict the returned value before execution, distinguish parameters from arguments, test with a case you can verify by hand, and explain the function's job in one sentence.

NAZARBAYEV UNIVERSITY

A good function is a small engineering promise

Its name states the job. Its parameters expose the required information. Its return value carries the result. Its tests and documentation justify trust.

ENG 101 - Programming for Engineers